CMP2103 Individual Member Revision Guide

Makerere University College of Engineering Design Art and Technology

Data Visualization Library in C++17. Reference main 22a2f81, verified 6 November 2026. Use this short guide after studying the complete Word handbook.

The whole project in one minute

DataLoader reads a header-bearing CSV into DataSet. DataPreprocessor::numericPairs extracts aligned finite X/Y vectors. Statistics::summarize returns a Summary from Y. The demo separately creates scatter, line and bar plots from the clean pairs. Each plot calculates local scale mappings and draws through Renderer. An Annotation adds a note, then ImageExporter writes SVG.

Verified demonstration

CSV pairs: (1,-2), (2,3), (3,1), (4,6), (5,4). Count 5, sum 12, mean 2.4, sorted median 3, population variance 7.44. Generated files: scatter.svg, line.svg and bar.svg.

All 13 registered tests passed in the final complete run and in a fresh build of merged main. No Windows security control was disabled. Passing the tests does not prove every input or behavior is correct.

What every member must know

[ ] Explain the data flow without inventing an inheritance chain.

[ ] Name your files, inputs, outputs and error behavior.

[ ] Explain one real source excerpt line by line.

[ ] Demonstrate a test and interpret its assertions.

[ ] State the difference between an assigned module and exclusive authorship.

AI assisted substantially with development, debugging and integration. Explain what you have reviewed and verified. Do not claim independent original authorship of code you cannot explain.

Jackson revision page

Assigned responsibility: Data Loading and Image Export. This is not a claim of sole authorship of all integrated code.

Files and main interfaces

DataLoader.h/.cpp, ImageExporter.h/.cpp, test_data_loading.cpp, test_image_export.cpp.

Explain these without notes

[ ] DataSet has public headers and rows. numericValue converts a field; it does not guarantee finiteness.

[ ] parseCSV tracks inQuotes and justClosedQuote. Doubled quotes represent one literal quote.

[ ] DataLoader is a static utility. ExportStrategy is abstract and SvgImageExporter overrides exportPlot.

[ ] preparePath checks nonempty filename and lowercase .svg suffix. The demo creates directories.

[ ] lastError_ is private. The named exporter exposes lastError(); the convenience class returns bool.

Fast oral questions

Why strings first? CSV may include nonnumeric fields. Numeric conversion happens when needed.

Why consumed from stod? It rejects trailing garbage such as 12kg.

Who calculates marker positions? The plot and its scale, not ImageExporter.

What does a false export mean? The path/format/write failed; inspect lastError on a named strategy.

Why a virtual destructor? A base pointer must destroy a derived exporter correctly.

Demonstration test selection

After the common build, run CTest with --test-dir .\build\behavior-repair --output-on-failure and -R '^(data_loading|image_export)$'. Run from the repository root.

Practice structure: my assigned module; its input and output; the essential code; a worked example; error behavior; its integration; the test evidence; one honest limitation.

Gladys revision page

Assigned responsibility: Data Preprocessing and Statistical Analysis. This is not a claim of sole authorship of all integrated code.

Files and main interfaces

DataPreprocessor.h/.cpp, Statistics.h/.cpp, test_preprocessing.cpp, test_statistics.cpp, test_integration.cpp.

Explain these without notes

[ ] numericPairs returns pair<vector<double>,vector<double>> and validates column indices.

[ ] Reject is the default. DropRow removes both coordinates to preserve alignment.

[ ] Statistics::summarize is a static member-function template, not a class template.

[ ] Summary stores count, sum, min, max, mean, median, variance and standardDeviation.

[ ] Variance uses m2/n. Median sorts a copy, leaving plot order unchanged.

Fast oral questions

Why reject infinity? It cannot map reliably to finite drawing coordinates.

Why drop both values? Dropping just X or Y would pair unrelated observations.

Population or sample variance? Population: divide by n. The demo gives 37.2/5 = 7.44.

Does Statistics feed the plots? No. It and plots separately consume cleaned data.

What types can the template accept? Numeric vector element types convertible to double, including the tested int and double cases.

Demonstration test selection

After the common build, run CTest with --test-dir .\build\behavior-repair --output-on-failure and -R '^(preprocessing|statistics|integration)$'. Run from the repository root.

Practice structure: my assigned module; its input and output; the essential code; a worked example; error behavior; its integration; the test evidence; one honest limitation.

Adrian revision page

Assigned responsibility: Scatter Plots and Rendering Coordination. This is not a claim of sole authorship of all integrated code.

Files and main interfaces

ScatterPlot.h/.cpp, Renderer.h/.cpp, test_scatter.cpp, test_renderer.cpp, test_plot_behavior.cpp.

Explain these without notes

[ ] ScatterPlot publicly inherits Plot. Coordinate vectors and markerRadius_ are private.

[ ] SVG circles use radius 5, blue fill #2563eb and dark outline #0f172a.

[ ] setMarkerRadius accepts only a positive finite value. marker_ controls ASCII only.

[ ] Renderer privately owns width_, height_ and elements_. Drawing appends SVG element strings.

[ ] Plots use local LinearScale. pixelX adds left; pixelY subtracts from bottom.

Fast oral questions

Why bottom minus scale? SVG Y grows downward, while a larger data Y should appear higher.

How are negative values handled? The normalized formula subtracts the actual data minimum.

Does draw save a file? No. It populates Renderer. Export saves the SVG later.

Why escape XML? Text and color attributes must not break SVG markup.

What happens to an invalid radius? The setter throws before assignment, preserving the old radius.

Demonstration test selection

After the common build, run CTest with --test-dir .\build\behavior-repair --output-on-failure and -R '^(scatter|renderer|plot_behavior)$'. Run from the repository root.

Practice structure: my assigned module; its input and output; the essential code; a worked example; error behavior; its integration; the test evidence; one honest limitation.

Ivan revision page

Assigned responsibility: Labels and Annotations. This is not a claim of sole authorship of all integrated code.

Files and main interfaces

Labels.h, src/Labels.cpp, test_labels.cpp, test_integration.cpp. include/dataviz/Labels.cpp is a separate legacy teaching program.

Explain these without notes

[ ] Label privately stores text_ and provides getText, setText and virtual render.

[ ] getText returns const string&. It cannot safely outlive the Label object.

[ ] AxisLabel inherits constructors and basic rendering. It adds no automatic rotation.

[ ] Annotation inherits Label and overrides render to add a radius-3 point and offset text.

[ ] Plot titles and axis labels are inherited strings, not owned Label objects.

Fast oral questions

Where does Annotation draw? At canvas pixels (x,y), with text at (x+8,y-8).

Why a base reference? Label& can dispatch to Annotation::render at runtime.

Does Label reject empty text? No. That validation is not implemented.

Where is XML escaping? Renderer::drawText handles it.

How does the current code differ from old diagrams? The integrated Annotation derives from Label and does not use a label ID association.

Demonstration test selection

After the common build, run CTest with --test-dir .\build\behavior-repair --output-on-failure and -R '^(labels|integration)$'. Run from the repository root.

Practice structure: my assigned module; its input and output; the essential code; a worked example; error behavior; its integration; the test evidence; one honest limitation.

Bonius revision page

Assigned responsibility: Axis Scaling. This is not a claim of sole authorship of all integrated code.

Files and main interfaces

AxisScale.h/.cpp, test_scaling.cpp. The similarly named .cpp under include is a separate teaching program.

Explain these without notes

[ ] AxisScale is abstract and stores private minValue and maxValue.

[ ] setRange rejects nonfinite limits and min greater than max.

[ ] LinearScale adds private screenWidth and requires a positive finite width.

[ ] scale(value) = (value-min)/(max-min) * width for a varying range.

[ ] A constant range maps to width/2. PercentageScale uses 100 and returns 50 for a constant range.

Fast oral questions

Can AxisScale be instantiated? No. scale is pure virtual.

Does scale include the left offset? No. It returns a distance. The plot adds left.

Why a midpoint for equal limits? It avoids division by zero and keeps points visible.

Does it clamp values? No. Values outside the range extrapolate.

Which scale does the demo use? LinearScale inside the plots, not PercentageScale.

Demonstration test selection

After the common build, run CTest with --test-dir .\build\behavior-repair --output-on-failure and -R '^scaling$'. Run from the repository root.

Practice structure: my assigned module; its input and output; the essential code; a worked example; error behavior; its integration; the test evidence; one honest limitation.

Sandra revision page

Assigned responsibility: Line Plots. This is not a claim of sole authorship of all integrated code.

Files and main interfaces

LinePlot.hpp, src/LinePlot.cpp, test_lineplot.cpp, test_line.cpp, example_lineplot.cpp.

Explain these without notes

[ ] LinePlot inherits Plot metadata and owns x_values_ and y_values_ privately.

[ ] setData checks equal lengths and finite values before storing new vectors.

[ ] segments returns consecutive Point pairs in data coordinates.

[ ] render maps points through local scales, then draws lines and points.

[ ] saveSvg is a separate richer SVG path, but it uses the same inherited title and labels.

Fast oral questions

Does it sort X? No. It preserves input order.

How many segments for five points? Four consecutive segments.

What about a singleton? It has no segments but can render a point.

What happens after invalid setData? Validation occurs first, preserving the old data in the tested cases.

What is a Point? A simple struct with public x and y fields, shared with Renderer.

Demonstration test selection

After the common build, run CTest with --test-dir .\build\behavior-repair --output-on-failure and -R '^(lineplot|line|plot_behavior)$'. Run from the repository root.

Practice structure: my assigned module; its input and output; the essential code; a worked example; error behavior; its integration; the test evidence; one honest limitation.

Jeff revision page

Assigned responsibility: Bar Plots. This is not a claim of sole authorship of all integrated code.

Files and main interfaces

BarPlot.h/.cpp, test_bar.cpp, test_plot_behavior.cpp, example_barplot.cpp.

Explain these without notes

[ ] BarPlot inherits Plot and privately stores categories and values.

[ ] addBar rejects nonfinite values, then appends the category and numeric value.

[ ] The numeric range includes zero even for all-positive or all-negative data.

[ ] SVG baseline is bottom - scale(0). Rectangle height is abs(y-baseline).

[ ] ASCII bars extend left for negative and right for positive values around a shared | column.

Fast oral questions

Why include zero? Otherwise bar height would not represent magnitude relative to zero.

Why min(y,baseline)? SVG rectangles need the upper-left corner regardless of sign.

Why draw the zero line last? It stays visible over the bars.

What about an all-zero dataset? The constant scale places its baseline at the midpoint.

What do getters return? Const references to the category and value vectors.

Demonstration test selection

After the common build, run CTest with --test-dir .\build\behavior-repair --output-on-failure and -R '^(bar|plot_behavior)$'. Run from the repository root.

Practice structure: my assigned module; its input and output; the essential code; a worked example; error behavior; its integration; the test evidence; one honest limitation.

OOP and language revision

Class and object ScatterPlot is a type. scatter in main is one instance with stored coordinates.

Encapsulation Private state plus public operations controls access. Radius validation preserves a positive finite invariant.

Protected access Plot shares title and label strings with derived plots. Ordinary callers use setters/getters.

Inheritance Plot has three chart subclasses. AxisScale, Label and ExportStrategy have their own verified hierarchies.

Runtime polymorphism Plot* selects a concrete virtual render implementation while the program runs.

Compile time generic programming Statistics::summarize<T> instantiates for the vector element type.

Abstraction A pure virtual contract states what the caller can request without requiring one drawing implementation.

Constructor Initializes base and member state. A derived object constructs its base first.

Destructor Standard containers release their own memory. Virtual base destructors support safe polymorphic destruction.

Const reference Allows read access without copying, but its lifetime depends on the referenced object.

Declaration and definition A declaration describes a name/signature. A definition supplies storage or implementation.

Compilation and linking Each production .cpp compiles separately. The linker combines object code and resolves references.

Do not call every function an OOP example. A numerical formula is ordinary computation. Its placement behind an interface may demonstrate abstraction.

Build and live demonstration commands

Run PowerShell from the repository root. The Ninja path below is machine-specific. Use a new build directory for an independent clean run. Do not bypass Windows Application Control if a new executable is blocked.

```
$env:PATH = "C:\msys64\ucrt64\bin;" + $env:PATH

Set-Location "C:\Users\GLADYS\Documents\DataVisualization_Final_Integration"

& "C:\Program Files\CMake\bin\cmake.exe" -S . -B build/behavior-repair -G Ninja `

"-DCMAKE_MAKE_PROGRAM=C:/Program Files/MATLAB/R2025b/toolbox/shared/coder/ninja/win64/ninja.exe" `

-DCMAKE_CXX_COMPILER=C:/msys64/ucrt64/bin/g++.exe -DCMAKE_BUILD_TYPE=Debug

& "C:\Program Files\CMake\bin\cmake.exe" --build build/behavior-repair -j 4

& "C:\Program Files\CMake\bin\ctest.exe" --test-dir build/behavior-repair --output-on-failure

.\build\behavior-repair\DataVisualization\csv_demo.exe ".\DataVisualization\data\demo.csv" ".\demo_output"
```

Expected output

Rows: 5 Mean: 2.4 Median: 3 Population variance: 7.44

Inspect demo_output/scatter.svg, line.svg and bar.svg in a browser or SVG viewer. Explain all five observations, the negative bar, the line order and the scatter circle radius.

Test names

bar, data_loading, image_export, lineplot, renderer, scaling, scatter, integration, statistics, preprocessing, labels, line, plot_behavior.

Final defense checklist

[ ] I can explain my module without claiming sole authorship.

[ ] I can identify private, protected and public members in a real header.

[ ] I can trace CSV data to the output files.

[ ] I can explain one validation condition and its exception or return value.

[ ] I can distinguish data flow, function calls and inheritance.

[ ] I can calculate mean 2.4, median 3 and population variance 7.44.

[ ] I can explain screen Y inversion and the bar zero baseline.

[ ] I can run a relevant test through CTest and state what it asserts.

[ ] I can distinguish a source defect from an executable blocked before launch.

[ ] I can explain AI assistance honestly and show what I independently checked.

Common mistakes to avoid

Do not claim PNG export, a GUI, automatic label placement, correlation or regression. Do not say Statistics supplies plot data. Do not treat legacy preprocessing inheritance as production architecture. Do not say AxisLabel automatically rotates text. Do not claim exportData writes a CSV. Do not call a blocked test a pass.

When you do not know an answer

State what you know, identify the source function and reason through its inputs and outputs. If uncertain, say so and inspect the implementation. A correct boundary on your knowledge is stronger than an invented feature or explanation.

Evidence

Source: merged main 22a2f81 at github.com/abitegekaivan256-oss/data-visualization-project. Assignments: TEAM_FILE_OWNERSHIP.md and individual reports. PR 11 merged integration-repair. Use the complete handbook and exact-source Markdown excerpts for detailed study.